

# Error Handling in Syntax Analysis

## 1. Introduction to Error Handling

### What Should a Compiler Do About Errors?

A good compiler must handle errors effectively by performing three main tasks:

#### 1. Detect Errors

- Identify when something is wrong in the source code
- Recognize violations of language rules

#### 2. Locate Errors

- Pinpoint exactly where the error occurred
- Provide line numbers and context

#### 3. Recover from Errors

- Continue parsing after finding an error
- Detect subsequent errors in the same compilation pass
- Avoid cascading false error messages

### Why Error Handling is Important

#### For Developers:

- Faster debugging
- Better understanding of mistakes
- More productive development

#### For Compiler Quality:

- Professional appearance
- User satisfaction
- Competitive advantage

## 2. Types of Errors in Compilation

Errors can occur in any of the three main analysis phases:

### Phase 1: Lexical Analysis

- Invalid characters
- Malformed tokens
- Illegal character sequences

## Phase 2: Syntax Analysis

- Grammar violations
- Missing or misplaced punctuation
- Incorrect program structure
- **Most errors are detected here!**

## Phase 3: Semantic Analysis

- Type mismatches
- Undeclared variables
- Scope violations
- Logical errors

**Focus of These Notes:** Syntax errors and their recovery strategies

## 3. Lexical Errors

### Common Lexical Errors

#### 1. Unexpected Character

`int x = 5@; // '@' is not valid in most languages`

#### 2. Illegal Sequence of Characters

`float f = 12.34.56; // Two decimal points`

`int x = 0x12G; // G is not a valid hex digit`

#### 3. Unterminated Strings

`char* str = "hello world; // Missing closing quote`

### Recovery Strategy for Lexical Errors

#### Simple Approach:

1. **Delete the offending character** from the input
2. **Continue scanning** as if nothing happened
3. **Report the error** to the user

#### Example:

Input: `int x@ = 5;`

Error: Unexpected character '@' at line 1, column 6

Action: Delete '@', continue with `"int x = 5;"`

## 4. Syntax Errors

### Why Syntax Errors Are Important

Most errors in programs are detected in the syntax analysis phase.

This is because:

- Syntax rules are strict and well-defined
- Parser checks every token sequence
- Structure violations are easy to detect

### Common Syntax Errors

#### 1. Missing Punctuation

```
if (x > 0)
```

```
    y = 5    // Missing semicolon
```

#### 2. Mismatched Delimiters

```
int arr[10;    // '[' not closed with ']'
```

```
func(a, b));  // Extra closing parenthesis
```

#### 3. Incorrect Structure

```
if x > 0 {    // Missing parentheses around condition
```

```
    y = 5;
```

```
}
```

### Requirements for Syntax Error Handling

A parser should:

1. ✓ **Detect all syntactic errors** (no error should go unnoticed)
2. ✓ **Report errors clearly and accurately** (helpful messages)
3. ✓ **Recover quickly** (minimal skipping of valid code)
4. ✓ **Detect subsequent errors** (find multiple errors in one pass)
5. ✓ **Not degrade compiler efficiency** (fast even with errors)

### The Main Difficulty

The main challenge is **balancing these competing objectives**:

- Thorough detection vs. Fast recovery
- Detailed messages vs. Efficiency

- Continuing vs. Stopping at first error

## 5. Error Recovery Strategies

There are four main strategies for recovering from syntax errors. Each has different tradeoffs.

### Strategy 1: Panic Mode Recovery

**Concept:** When an error is found, **ignore/skip all symbols** until a "synchronizing" token is found.

#### Synchronizing Tokens:

- Delimiters like `;`, `}`, `end`, `)`
- Keywords that start statements: `if`, `while`, `return`
- Tokens that clearly indicate a new construct

#### Algorithm:

When error is detected:

1. Pop symbols from stack until synchronizing state
2. Skip input symbols until synchronizing token
3. Resume parsing from that point

#### Example:

```
if (x > 0
    y = 5; // Error: missing ')'
    z = 10;
}
```

Recovery:

- Detect error at `'y'`
- Skip until `';'` (synchronizing token)
- Resume parsing at `'z = 10;'`

#### Advantages:

- ✓ **Simple to implement**
- ✓ **Guaranteed to terminate** (always makes progress)
- ✓ **Works for most errors**

**Disadvantages:**

- ✗ **Ignores a lot of code** (may skip valid statements)
- ✗ **May miss errors** in skipped region
- ✗ **Coarse recovery** (not precise)

**Strategy 2: Phrase-Level Recovery**

**Concept:** Replace, insert, or delete a small portion of the input to allow the parser to continue.

**Common Actions:**

- **Insert** a missing delimiter (;, ), }
- **Delete** an extra delimiter
- **Replace** one token with another

**Algorithm:**

When error is detected:

1. Analyze the context
2. Determine likely cause
3. Make minimal local change
4. Continue parsing

**Example 1: Missing Semicolon**

Input: `x = 5 y = 10;`

Error: Expected ';' after '5'

Action: Insert ';'

Result: `x = 5; y = 10;`

**Advantages:**

- ✓ **Precise recovery** (minimal code skipped)
- ✓ **Handles common errors** well
- ✓ **Good error messages** possible

### Disadvantages:

- ✗ **Danger of infinite loop** (if changes don't help)
- ✗ **Limited scope** (can't fix errors on stack)
- ✗ **Complex to implement** properly
- ✗ **May make wrong assumption** about error

### Strategy 3: Error Productions

**Concept:** Add extra productions to the grammar that recognize commonly occurring errors.

#### How It Works:

1. Analyze typical programming errors
2. Add grammar productions for these patterns

#### Example Grammar with Error Productions:

##### Original Grammar:

```
stmt → if ( expr ) stmt  
      | expr ;
```

##### With Error Productions:

```
stmt → if ( expr ) stmt  
      | if expr stmt      /* missing parentheses */  
      | if ( expr stmt    /* missing closing paren */  
      | expr ;  
      | expr              /* missing semicolon */
```

##### Error Message Examples:

if expr stmt → "Missing parentheses around condition"

if ( expr stmt → "Missing ')' in if statement"

expr → "Missing semicolon after statement"

##### Complete Example:

Input: if x > 0 { y = 5; }

Without error productions:

- Multiple errors reported
- Confusing recovery

With error productions:

- Matches: if expr stmt
- Message: "Missing parentheses around condition in if statement"
- Clean recovery

**Advantages:**

- ✓ **Specific error messages** (knows exact problem)
- ✓ **Targeted recovery** (handles specific cases well)
- ✓ **Ensures good recovery** for common errors
- ✓ **Professional appearance**

**Disadvantages:**

- ✗ **Requires analysis** of language usage patterns
- ✗ **Grammar becomes larger** (more productions)
- ✗ **Maintenance overhead** (keep error productions updated)
- ✗ **May cause conflicts** in parser generation

**Strategy 4: Global Correction**

**Concept:** Determine the minimum number of changes needed to make the entire program syntactically correct.

**How It Works:**

1. Find all possible corrections
2. Calculate cost of each correction
3. Choose minimum-cost correction
4. Apply changes globally

**Algorithm Approach:**

- Uses dynamic programming
- Considers insertions, deletions, substitutions
- Finds optimal edit distance

**Example:**

Input: `int x = 5 + ;`

Possible corrections:

1. Insert '0' after '+': `int x = 5 + 0;` (1 insertion)
2. Delete '+': `int x = 5 ;` (1 deletion)
3. Replace '+' with valid value: ... (1 substitution)

**Advantages:**

- ✓ **Theoretically optimal** (minimum changes)
- ✓ **Mathematically sound** algorithms exist
- ✓ **Best possible correction**

**Disadvantages:**

- ✗ **Extremely expensive** in time and space
- ✗ **Requires complete program traversal**
- ✗ **Impractical for large programs**
- ✗ **May not match programmer's intent**
- ✗ **Slows down compilation significantly**

## 6. Error Handling in LR Parsing

LR parsers (like LALR used in yacc) have specific error handling mechanisms.

**Error Detection in LR Parsing****Detection:**

- LR parsers detect errors at the **earliest possible point** in the input
- An error is detected when reaching an **error entry** in the parse table
- No valid action exists for current state and input symbol

**Example:**

Parse Table Entry: [State 5, Input '+'] = ERROR

Meaning: In state 5, seeing '+' is syntactically invalid

**Panic Mode in LR Parsing****Algorithm:****Step 1: Scan Down Stack**

- Look down the stack until you find a state  $s$
- State  $s$  must have a shift entry  $S_i$  for one of a set of particular non-terminals  $A$
- Common choices for  $A$ : stmt, expr, block

**Step 2: Discard Input**

- Throw away input symbols
- Continue until a symbol from FOLLOW( $A$ ) is reached
- FOLLOW( $A$ ) typically contains delimiters like  $;$ ,  $\}$ ,  $)$

**Step 3: Modify Stack**

- Discard all states above  $s$  from stack
- Place state  $i$  on stack
- Pretend that an  $A$  was successfully parsed

**Step 4: Resume**

- Continue parsing from the new configuration

**Rationale:**

- Assumes error occurred in a derivation from non-terminal  $A$
- Pretends that  $A$  was correctly parsed
- Synchronizes at statement or expression boundaries

**Example:**

Input:  $x = 5 + ; y = 10;$

↑

Error here

Stack: [State representing:  $x = 5 +$ ]

Recovery:

1. Scan stack for state with entry for 'stmt'
2. Discard input until ';' (in FOLLOW(stmt))
3. Pop stack states, push state for 'stmt'
4. Resume at 'y = 10;'

**Visual:**

Before Error:

Stack: [0, expr, 3, +, 7]    Input: ; y = 10;

↑ Error

After Panic Mode:

Stack: [0, stmt, 2]    Input: y = 10;

↑ Resume here

### **Phrase-Level Recovery in LR Parsing**

**Algorithm:**

#### **Step 1: Create Error Procedures**

- For each error entry in the parse table
- Insert a pointer to a specific error procedure
- Each procedure assumes the most likely cause

#### **Step 2: Error Procedure Actions**

- Diagnose the specific error
- Make appropriate correction (insert/delete/replace)
- Issue helpful error message
- Continue parsing

### Example Parse Table with Error Procedures:

State | a + \* ( ) \$

-----+-----

0 | S1 e1 e1 S2 e1 e1

1 | e3 S6 e4 e3 e2 e3

2 | S1 e1 e1 S2 e5 e1

3 | e3 e2 S7 e3 e2 e3

...

### Error Procedure Examples:

#### e1: Missing Operand

Situation: Expecting 'a' or '(', but finding '+', '\*', or '\$'

Action: Push state 5 on stack (assume 'a')

Message: "Missing operand"

#### e2: Unmatched Right Parenthesis

Situation: Finding unexpected ')'

Action: Remove ')' from input (ignore it)

Message: "Unmatched right parenthesis"

#### e3: Missing Operator

Situation: Expecting '+', finding 'a' or '('

Action: Push state 6 on stack (assume '+')

Message: "Missing operator '+'"

#### e4: Wrong Operator

Situation: Expecting '+', finding '\*'

Action: Push state 6 on stack (assume '+')

Remove '\*' from input

Message: "'\*' used instead of '+'"

#### e5: Missing Right Parenthesis

Situation: Expecting ')', finding '\$'

Action: Push state 11 on stack (assume ')')

Message: "Missing right parenthesis"

### Complete Example:

Expression:  $a + * a$

Parse trace:

| State Stack | Input | Action |
|-------------|-------|--------|
|-------------|-------|--------|

|     |              |                    |
|-----|--------------|--------------------|
| [0] | $a + * a \$$ | Shift 1 (read 'a') |
|-----|--------------|--------------------|

|        |            |                    |
|--------|------------|--------------------|
| [0, 1] | $+ * a \$$ | Shift 6 (read '+') |
|--------|------------|--------------------|

|           |          |          |
|-----------|----------|----------|
| [0, 1, 6] | $* a \$$ | ERROR e4 |
|-----------|----------|----------|

- Message: "'\*' used instead of '+'"

- Push state 6 (assume '+')

- Remove '\*'

|              |        |                    |
|--------------|--------|--------------------|
| [0, 1, 6, 6] | $a \$$ | Shift 1 (read 'a') |
|--------------|--------|--------------------|

|                 |      |           |
|-----------------|------|-----------|
| [0, 1, 6, 6, 1] | $\$$ | Reduce... |
|-----------------|------|-----------|

## 7. Yacc Error Recovery

**Yacc** (Yet Another Compiler Compiler) uses a special approach similar to error productions.

### The error Token

Yacc provides a special token called error that can be used in grammar productions.

### Syntax:

$A \rightarrow \text{error } \alpha$

Where:

- error is a special token representing "any erroneous input"
- $\alpha$  is a synchronizing pattern (usually a delimiter)

### How Yacc Error Recovery Works

#### Algorithm:

#### Step 1: Error Detection

- Error is found during parsing

#### Step 2: Scan Down Stack

- Scan down the stack until finding a state
- State must have a production of form:  $A \rightarrow \cdot \text{error } \alpha$
- This means: we can recover by treating this as part of A

### Step 3: Scan Through Input

- Scan through input looking for a string matching  $\alpha$
- $\alpha$  is typically a synchronizing token like  $;$ ,  $\}$ , end

### Step 4: Discard Input

- Remove all input up to and including the match for  $\alpha$

### Step 5: Reduce

- Parser reduces to A
- Continue normal parsing

### Example: Statement Error Recovery

#### Grammar:

stmt: expr ';'

| if '(' expr ')' stmt

| '{' stmt\_list '}'

| error ';' /\* Error recovery production \*/

;

#### How it works:

Input: x = 5 + ; y = 10;

↑

Error

#### Recovery:

1. Error detected at incomplete expression
2. Stack has state with:  $\text{stmt} \rightarrow \cdot \text{error } ';'$
3. Scan input for  $';'$
4. Found  $';'$  after incomplete expression
5. Discard "x = 5 + ;"

6. Reduce to: stmt  $\rightarrow$  error ';'

7. Continue with "y = 10;"

### Visual:

Before:

Input: x = 5 + ; y = 10;

↑

ERROR

Match  $\alpha$ : ↓

Input: x = 5 + ; y = 10;

↑-----↑

Discard this part

After:

Reduced to: stmt

Continue: y = 10;

### Multiple Error Productions

#### Grammar with Multiple Recovery Points:

program: stmt\_list

;

stmt\_list: stmt

| stmt\_list stmt

;

stmt: expr ';'

| if '(' expr ')' stmt

| if '(' expr ')' stmt else stmt

| while '(' expr ')' stmt

```

| '{ stmt_list }'
| error ';' /* Recover at statement end */
| if error ')' stmt /* Recover at if condition */
| while error ')' stmt /* Recover at while condition */
;

```

expr: term

```

| expr '+' term
| error /* Recover in expression */
;

```

### Benefits:

- Multiple recovery points
- Different granularity of recovery
- More specific error messages

### Advantages of Yacc Error Recovery

✓ **Integrated with grammar** (part of language definition) ✓ **Flexible** (can have multiple error productions) ✓ **Automatic** (yacc generates recovery code) ✓ **Customizable** (can add actions to error productions)

### Example with Actions

```

stmt: error ';' {
    yyerror("Syntax error in statement");
    yyerrok; /* Tell yacc error is handled */
}
;

```